

One Person Lab

One Person Lab 白皮书

让一个人也能拥有专业团队级的复杂知识工作推进能力

One Person Lab 让 AI 从一次回答走向可复查、可交付、可持续推进的专业工作。

OPL Framework / One Person Lab App / OPL Cloud / Foundry Agents

2026-07-13

Public whitepaper

目录

1	从生成走向完成	2
2	OPL 的答案：把 AI 组织成一支可接力的专业团队	2
3	认知计算：在阶段内完成真正的专家工作	2
4	三层产品：一个体验，清楚分工	4
5	三类机制：用户为什么会觉得它好用	4
6	标准专业智能体：声明能力，保留权威	5
7	一个成果如何从想法走到可交付	5
8	五条设计原则	5
8.1	AI-first, contract-light	5
8.2	交付即推进	5
8.3	目标先于路径	6
8.4	真相归主	6
8.5	抓大放小	6
9	简单维护：用户只需要认识三个对象	6
10	为什么用户可以相信 OPL	6
11	结语	6

发布日期：2026-07-13

最近修订：2026-08-10

适用对象：希望理解 One Person Lab 为什么这样设计，以及这种设计怎样让复杂知识工作更好用、更可信的用户、合作者、早期采用者和技术决策者。

核心判断：One Person Lab 让 AI 从一次回答走向可复查、可交付、可持续推进的专业工作。

1 从生成走向完成

AI 已经很擅长回答一个问题、生成一段代码或润色一份材料。但论文、基金申请、正式汇报、书稿这类工作，很少在一次对话里结束。它们跨越数天、数周甚至更久，中间要持续补材料、比较路线、修改文件、接受审阅、修正错误并做出关键决定。

真正困难的是让每一次生成都进入一条持续完成、可以复查的工作线：

- 多轮工作之后，当前究竟推进到了哪里？
- 哪些材料和证据支撑了现有判断，最新版成果在哪里？
- 谁在执行，谁在独立审阅，什么结论仍需负责人确认？
- 出现质量问题时，怎样继续形成有用成果，同时如实标记完成边界？
- 用户离开后，工作怎样保留线索；回来时，又怎样立即知道卡点和下一步？

对话工具擅长处理当下问题，项目管理工具擅长登记任务，固定自动化擅长执行确定步骤。复杂知识工作还需要一套新的工作方式：让 AI 有足够空间做专家判断，同时让阶段、文件、证据、责任和交付边界始终清楚。

这就是 One Person Lab (OPL) 要解决的问题。

2 OPL 的答案：把 AI 组织成一支可接力的专业团队

OPL 从最终成果反推工作结构，把复杂任务组织成一系列能产生真实增量、又能根据证据灵活调整的阶段：准备材料、形成方案、执行创作、独立审阅、修订完善、交付收口。

每个阶段都围绕可检查的成果工作。AI 执行者可以阅读材料、比较方案、调用工具、创作内容并根据反馈继续修订；独立审阅者从不同角色检查证据、逻辑、质量和交付风险；系统则保存阶段状态、文件、证据引用、责任方、卡点和恢复线索。

用户直接感知到：

- 工作持续向下一版成果推进，每轮对话都沉淀为可继续的增量；
- 每一版都能打开、检查、修改和继续交接；
- 质量差距被明确指出，状态始终与实际进展一致；
- 需要人做决定时，界面直接说明“谁需要做什么”。

OPL 用清楚的边界释放 AI。开放式判断交给 AI，权限、证据、责任和恢复交给系统。

3 认知计算：在阶段内完成真正的专家工作

固定自动化适合“先做 A，再做 B，最后输出 C”。复杂知识工作却经常需要反复理解、比较、推翻、重写和审阅。论文的论证路线会因为新证据改变，基金申请的创新点会在模拟评审后重构，正式汇报的故事线也会在视觉呈现中被重新组织。

OPL 把这种阶段内的开放式专家工作称为**认知计算**：AI 在一个目标清楚、边界可见、结果可接力的阶段里完成理解、比较、创作、审阅和修订。

这种设计有两个重要结果。

第一，AI 执行者越强，阶段内的真实工作能力就越强。模型、提示词、专业技能和领域知识的进步，可以直接提升理解、判断和创作质量，让专家思路在开放的阶段中充分发挥。

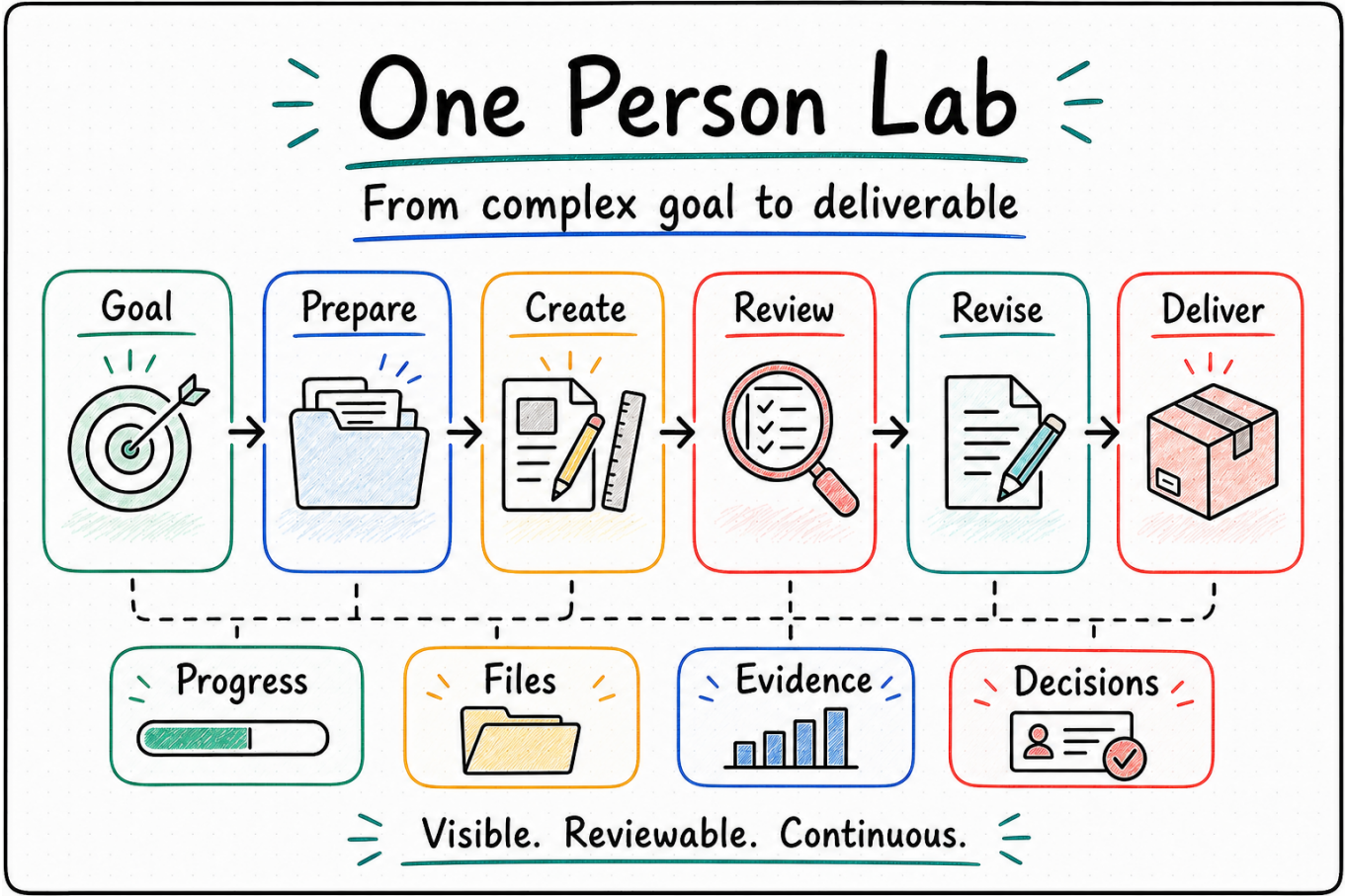


图 1: One Person Lab 把复杂目标组织为可见、可复查、可持续推进的成果链。

第二，能力提升与可控运行同步发生。OPL 让每次阶段推进都有材料范围、权限边界、预期成果、证据引用、独立审阅和交接要求。AI 可以自由决定如何完成工作，最终结论始终由明确的责任方给出。

因此，OPL 的基本协作单元是一次有目标、有产物、有审阅、有去向的专业工作尝试。

4 三层产品：一个体验，清楚分工

One Person Lab 对用户呈现为一个连续产品，对内保持三层分工，分别为可靠运行、易用界面和专业判断指定清楚的责任方。

OPL Framework 是运行底座。它负责显式启动、阶段推进、长期运行、恢复、工作空间、证据与交接，让复杂任务可以被启动、暂停、继续和收口。当前第一公民执行者是 Codex CLI；生产级在线运行以 Temporal-backed provider 为必需底座。

One Person Lab App 是当前用户工作面，通过 desktop 与 Docker/WebUI 让用户选择任务、查看进度、打开文件、处理卡点和接收更新。**OPL Cloud** 是长期、条件启用的产品包装：只有真实 account、storage、isolation、backend 与 owner policy 齐备时，才把同一条工作线扩展到在线工作空间、托管资源、组织管理和协作场景。二者都沿用 Framework 与领域 owner 持有的运行事实和专业结论；Cloud 叙事不构成当前 release、runtime 或 production-ready 证明。

Foundry Agents 是专业能力层。不同领域需要不同材料理解、工作方法、质量标准和交付权威。OPL 当前的五个标准 Foundry Agents 是：

- **Med Auto Science (MAS)**：医学研究与论文交付；
- **Med Auto Grant (MAG)**：基金方向、申请书、模拟评审与修订；
- **RedCube AI (RCA)**：**Visual Deliverable Foundry**，负责演示文稿、报告、叙事、渲染、审阅与导出；
- **OPL Meta Agent (OMA)**：理解新建、接管和改进目标，形成智能体蓝图、评测规格和基于证据的演进建议；
- **OPL Book Forge (OBF)**：面向书籍与长篇手稿的策划、写作、审阅与交付。

五者统一使用 OPL 的标准运行、分发和工作面，但专业判断与最终权威仍然归属清楚。OMA 负责智能体的设计语义；测试执行、候选物化、独立评估、不可变版本、激活和回滚由 OPL Foundry Kernel 承担；保护测试正文、最终验收、权限授权和生产采用仍由目标 owner 决定。

三层之间有一条简单原则：Framework 负责“工作怎样可靠推进”，App（以及条件启用的 Cloud）负责“用户怎样看见并操作”，Foundry Agent 负责“这个专业领域什么算好、什么可以交付”。

5 三类机制：用户为什么会觉得它好用

OPL 内部有运行、工作空间、能力目录、证据、控制台和连接等多个模块。用户只需理解这些能力最终形成的三类可感知机制。

用户感知的机制	背后的设计	带来的实际体验
工作始终围绕下一份成果推进	阶段式认知计算；执行者与独立审阅者分离；可消费成果允许带着明确质量债进入下一轮	用户持续拿到可打开、可比较、可修改的版本，并能看见任务的实际进展
专业能力即装即用，判断权归属清楚	Declarative Domain Pack + OPL generated/hosted surfaces + minimal authority：领域用声明式 Pack 描述阶段与能力，OPL 生成和托管通用工作面，领域仓集中持有需要专业裁决的最小权威函数	MAS、MAG、RCA 等使用同一套入口、进度和文件体验，同时仍由真正懂该领域的 owner 给出质量与交付判断
工作可以停下、解释、恢复和交接	工作空间保存材料与产物，运行层保存尝试与恢复线索，证据层只引用真实来源，工作台优先显示当前责任方和下一步	用户离开后仍保有完整上下文；失败时能知道原因；换人或换阶段时能从已有成果继续

这三类机制共同解决一个常见矛盾：专业工作需要开放判断，而长期交付需要稳定秩序。OPL 让开放判断与稳定秩序同时成立。

6 标准专业智能体：声明能力，保留权威

传统做法往往让每个专业智能体各自开发一套运行器、队列、状态页、文件系统和安装方式。短期看起来灵活，长期却会让用户面对多个入口、多个更新器和多套互相冲突的状态。

OPL 的目标形态更简单：一个标准专业智能体由三部分组成。

1. **Declarative Domain Pack**: 声明这个领域有哪些阶段、专业能力、输入输出和质量边界。
2. **OPL generated/hosted surfaces**: 由 OPL 生成并托管通用入口、运行、进度、文件、恢复、安装和工作台能力。
3. **Minimal authority functions**: 领域仓集中持有需要专业裁决的最小权威动作，例如领域质量结论、产物授权、负责人回执或明确卡点。

共同的运行外围由 OPL 上收，每个 Agent 因此可以把更多精力放在自己的专业差异上。医学研究按证据与发表标准判断，基金按创新性与可资助性审阅，视觉交付按叙事、可读性与导出结果验收。

对用户而言，这意味着学习一次，就可以使用多个专业 Agent；对产品而言，这意味着各领域可以直接共享运行、恢复、文件和界面升级；对专业责任而言，“质量合格”始终由领域 owner 宣布。

7 一个成果如何从想法走到可交付

下面用一个医学研究项目说明这套设计。这个故事表达 OPL 的工作方式，每项研究可以根据自身需要调整具体步骤。

一位研究者把数据说明、研究问题、已有图表和参考文献放进工作空间，希望形成一篇可以继续投稿准备的论文初稿。

第一步，形成可工作的研究上下文。 MAS 读取材料，识别研究问题、数据限制和需要补齐的证据。OPL 保存材料位置、当前目标和阶段边界。研究者直接看到一份可确认的材料摘要、缺口和下一步，内部扫描日志留在诊断层。

第二步，执行者形成成果增量。 AI 执行者围绕研究问题组织分析结果、图表解释和稿件结构，产出新的稿件与证据引用。它可以自主比较路线、调整论证和调用工具；OPL 为思考路径与工具选择保留充分的自主空间。

第三步，独立审阅者提出异议。 reviewer 从独立视角检查数字、证据、论证和发表风险。它发现一项次要结论的证据仍不充分，但主结果、方法和稿件骨架已经可以支持下一轮工作。

第四步，有成果就继续，ready 边界保持清楚。 系统把这一阶段记录为 `completed_with_quality_debt`：可消费成果已经形成，后续修订可以继续；同时，证据不足被清楚保留为质量债。这个状态表示成果可供下一轮使用；`publication-ready`、MAS 质量结论与研究负责人确认仍走各自的权威路径。

第五步，修订并交回真正的责任方。 下一轮针对质量债补证据、收紧 claim、更新稿件。MAS 的领域质量门和研究负责人决定是否接受当前版本、继续修订或进入投稿准备。OPL 负责把文件、证据、审阅意见和责任流转完整交到他们手里。

用户最终获得一条可以复查的成果链：材料有来源，版本有位置，审阅有独立角色，问题有去向，最终结论有真正的责任方。

8 五条设计原则

OPL 的好用来自一组稳定设计判断。它们约束系统如何取舍，也解释为什么界面、运行和专业 Agent 会呈现为现在的样子。

8.1 AI-first, contract-light

AI 做专家工作，合同守住下限。

理解、判断、创作、审阅和修订优先交给 AI。合同聚焦身份、权限、安全动作、证据、卡点、恢复和交接，为工具顺序与创作策略保留开放空间。这样，更强的模型、更好的技能和更成熟的领域知识可以直接转化为更好的成果。

8.2 交付即推进

有可接力成果，就推动工作向前。

真实进度以成果、证据、决定和交接为依据。若阶段已经形成可消费成果，剩余质量差距可以在安全和权威边界内继续修订，OPL 允许以 `completed_with_quality_debt` 推进，并保留问题和 owner 路径。带质量债完成表示已有可继续使用的成果；领域质量、发布和生产就绪分别由对应权威继续证明。

8.3 目标先于路径

先问要交付什么，再决定系统怎样工作。

论文、基金、视觉交付和书稿需要不同专业路线。目录、状态、运行器和工具都服务于用户目标。与成果、证据、责任或交接无关的复杂面，应当收薄、下沉或退役。

8.4 真相归主

谁拥有事实，谁给结论。

Framework 说明运行是否发生、文件和证据在哪里；App 说明用户当前能看见和操作什么；Foundry Agent 和独立 reviewer 负责领域质量判断；真正的 owner 负责需要授权的最终决定。投影、测试和文档统一引用这些 owner source。

8.5 抓大放小

把硬门留给关键边界。

启动安全、权限、不可逆动作、必要人工确认、owner receipt 和权威边界必须严格保护；普通提示、诊断、评分和改进建议保持轻量并服务当前工作。系统守住大边界，AI 才能在阶段内充分工作。

9 简单维护：用户只需要认识三个对象

强大的系统把内部复杂性留在内部。OPL 把安装、更新和修复收敛为三个用户对象：

- **OPL Base**：Framework、Codex 执行路径、在线运行底座和通用能力；
- **OPL App**：可选的桌面工作台与产品体验；
- **OPL Packages**：MAS、MAG、RCA、OMA、OPL Book Forge、工作流与能力包。

用户只需维护 Base、App 和 Packages。Base 管共同底座，App 管桌面体验，Packages 管专业能力；领域事实和质量管理权威仍留在对应 owner。这样的运维设计让“专业能力很多”与“日常维护简单”可以同时成立。

10 为什么用户可以相信 OPL

信任来自系统持续回答几个关键问题：要交付什么，推进到哪里，当前成果在哪里，依据是什么，谁负责下一步，什么判断仍在等待授权。

OPL 用克制建立这种信任：

- **运行与质量分别证明**。阶段结束、测试通过或文件生成说明运行进展，领域质量结论由专业 owner 给出。
- **执行与终审角色分离**。复杂质量判断进入独立 reviewer 或领域 owner 路径。
- **成果及时呈现**。可消费成果继续推进，质量债同步记录并进入修复路径。
- **界面优先呈现行动信息**。默认界面先显示成果、责任方、卡点和下一步，技术细节在需要时展开。
- **专业权威归领域 Agent**。OPL 负责承载、连接和投影，领域 Agent 负责真正的专业判断。

这套边界让 OPL 既有能力长期运行，也能如实说明 ready 仍需哪些证明。用户得到一支会持续交付、接受审阅、暴露问题并把决定交回正确责任方的 AI 专业团队。

11 结语

One Person Lab 的长期目标，是让一个人也能拥有过去只有专业团队才能提供的持续推进能力。

AI 负责理解、比较、创作、审阅和修订；OPL Framework 负责阶段、工作空间、证据、恢复与交接；App 把这些能力变成当前清楚、可操作的 desktop + Docker/WebUI 工作体验，条件启用的 Cloud 可以在 owner prerequisites 齐备后扩展这一体验；MAS、MAG、RCA、OMA 与 OPL Book Forge 五个标准 Foundry Agents 负责各自的专业判断，并把最终交付或采用决定交回相应 owner。

OPL 相信，真正优秀的 AI 产品既能在用户提出问题时给出高质量回应，也能在漫长、复杂、充满反复的正式工作中，始终让下一份成果更近，让每个判断有依据，让每个问题有去向，让最终交付值得相信。

了解更多：

- [One Person Lab](#)
- [One Person Lab App](#)
- [OPL Cloud](#)
- [OPL Framework 白皮书在线版](#)